4-bit Array Multiplier: RTL to GDSII

DVD Assignment

Nikhil Venkat Atkuru(2024102039) Abhinav Venkata Kota(2024102059)

October 26, 2026

Contents

1 Part A - RTL Design and Verification **3**
1.1 Block Diagram . . . 3
1.2 Simulation Results . . . 3
1.2.1 PASS Summary Line . . . 4
1.2.2 Error Injection Test (FAIL Line) . . . 4
1.3 Hand-Worked Example . . . 5
1.4 Waveform with 2-Cycle Latency . . . 5

2 Part B - Logic Synthesis with Yosys **6**
2.1 Synthesis Results . . . 6
2.2 Gate-Level Netlist Simulation . . . 6
2.3 Report Questions . . . 6
2.3.1 Q1: Logic Cell Count vs. 16 AND Gates + 12 Full Adders . . . 6
2.3.2 Q3: Flip-Flop Share of Total Cell Area . . . 7

3 Part C - Static Timing Analysis with OpenSTA **7**
3.1 Finding the Maximum Frequency . . . 7
3.1.1 Run at $T = 10\,\text{ns}$ . . . 7
3.1.2 Run at $T = T_{\min}$ . . . 8
3.2 Timing Results . . . 8
3.3 Critical Path . . . 8
3.3.1 Start Point and End Point . . . 8
3.3.2 Cells on the Critical Path . . . 9
3.3.3 Comparison with the $2N$ Full-Adder Estimate . . . 9
3.4 Report Questions . . . 9
3.4.1 Q2: Critical Path and the $2N$ Estimate . . . 9

4 Part D - Physical Design: RTL to GDSII **10**
4.1 Flow Setup . . . 10
4.2 Timing Closure: Run History . . . 10
4.3 Physical Design Results . . . 11
4.4 Post-Route Maximum Frequency . . . 11
4.5 Cells Added by Place and Route . . . 11
4.6 Layout Screenshots . . . 12
4.7 Report Questions . . . 13
4.7.1 Q4: Post-Route $F_{\max}$ and Added Cells . . . 13
4.7.2 Q5: DRC and LVS . . . 14

5 Q6 - Scaling to an 8-bit Multiplier **14**

6 Contributions **14**

7 Github Repo **15**

1 Part A - RTL Design and Verification

1.1 Block Diagram

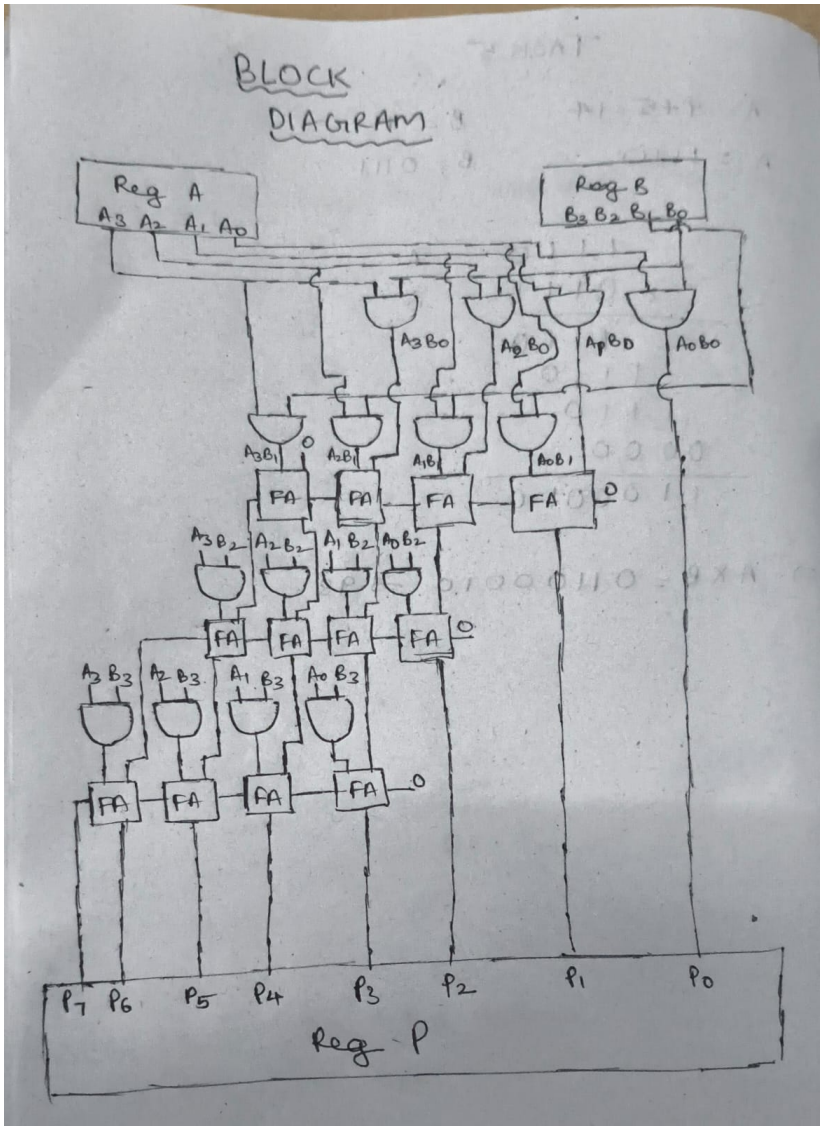


Figure 1: Block diagram of the 4-bit multiplier: input registers, 16 AND gates, 12 full adders and the output register

The inputs `a` and `b` are first stored in two 4-bit registers (Reg A and Reg B). The 16 AND gates form the partial products, the three rows of 4 full adders (12 in total) add them row by row, and the 8 product bits are stored in the output register (Reg P).

1.2 Simulation Results

The testbench applies all 256 input combinations ($a = 0 \ldots 15$, $b = 0 \ldots 15$), waits for the 2-cycle latency, and compares the output p with the expected value $a \times b$. It prints one line for every mismatch and a single summary line at the end. The subsections below show the output for the pass case and the fail case.

1.2.1 PASS Summary Line

With the correct full adder ($sum = a \oplus b \oplus c_{in}$), all 256 vectors match the expected product and no mismatch lines are printed:

```
PASS: 256 vectors checked, 0 errors
```

Simulation ends at $t = 5140$ ns. Reset takes the first 20 ns (2 clock cycles of 10 ns), and each of the 256 vectors takes 2 clock cycles (20 ns) because of the latency: $20 + 256 \times 20 = 5140$ ns.

1.2.2 Error Injection Test (FAIL Line)

To confirm that the testbench actually catches errors, the sum output in `rtl/full_adder.v` was temporarily changed from $a \oplus b \oplus c_{in}$ to $a \oplus b$ (the carry-in is ignored). The testbench was then re-run, and the change was reverted afterwards. The first and last mismatch lines and the summary line of the failed run are:

```
MISMATCH: a=3 b=3 p=5 expected=9 (t=1060000)
MISMATCH: a=3 b=6 p=10 expected=18 (t=1120000)
MISMATCH: a=3 b=7 p=9 expected=21 (t=1140000)
...
MISMATCH: a=15 b=15 p=181 expected=225 (t=5140000)
FAIL: 256 vectors checked, 95 errors
```

95 of the 256 vectors fail. The faulty sum is wrong only when a full adder receives a carry-in of 1. So every product that needs no carries inside the array still comes out correct: for example, any case with a or b equal to 0, 1, 2, 4 or 8. Every product that does generate a carry is caught. This shows that the testbench detects the injected fault.

1.3 Hand-Worked Example

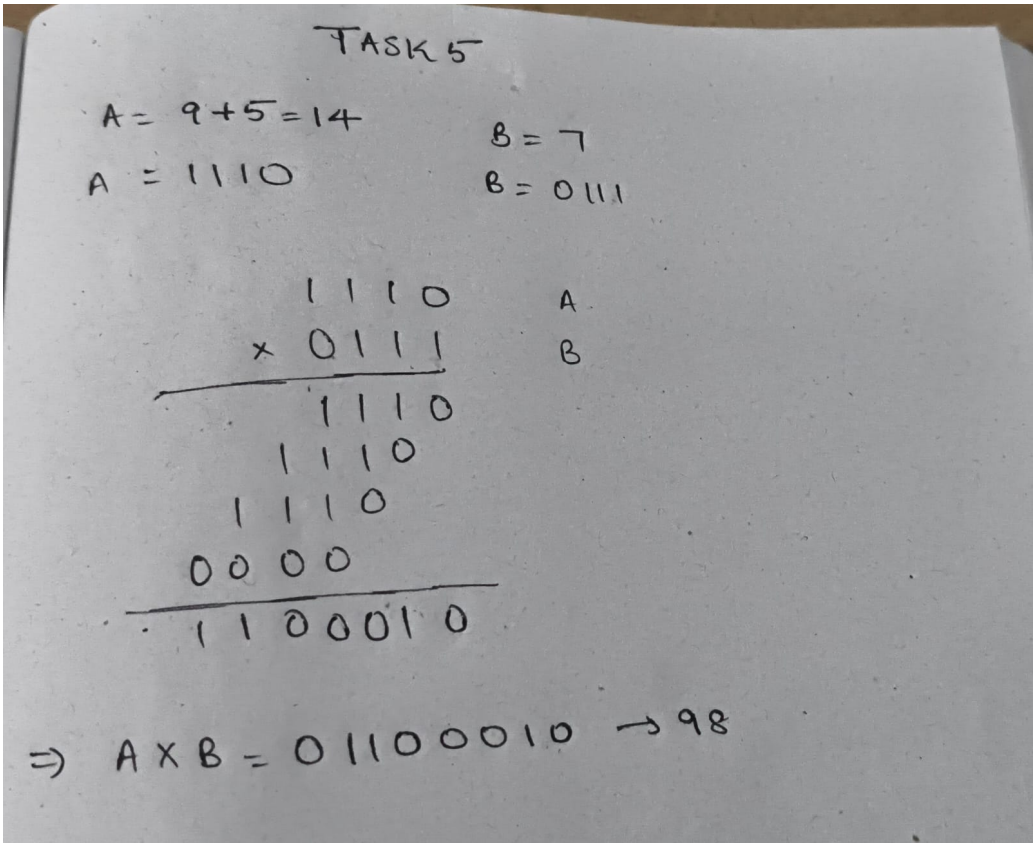


Figure 2: Hand-worked example: $14 \times 7 = 98$

Using the last digit of roll number 2024102059, $A = 9 + 5 = 14$ (`1110`) and $B = 7$ (`0111`). The four partial products (`1110`, `1110`, `1110` and `0000`), each shifted one place further left, add up to `01100010` $= 98 = 14 \times 7$.

1.4 Waveform with 2-Cycle Latency

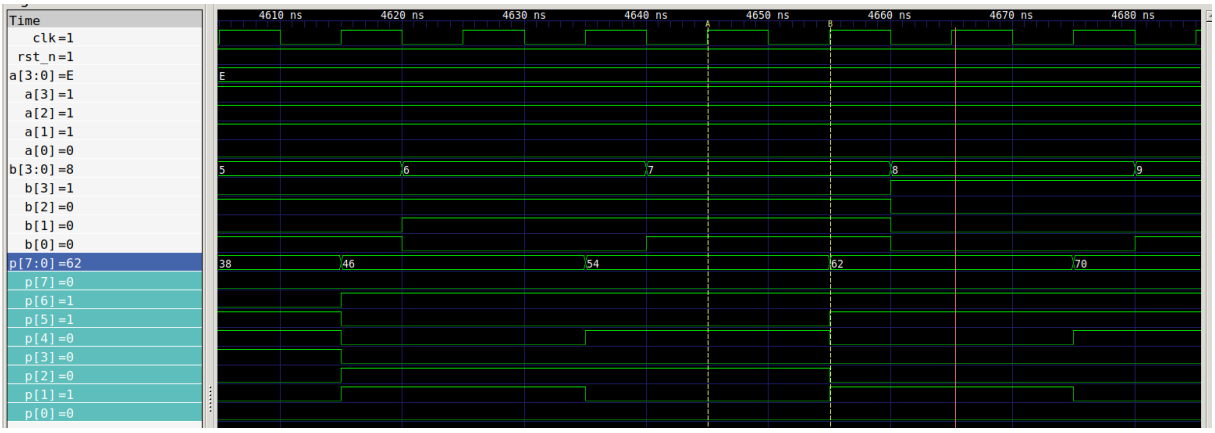


Figure 3: Waveform of `clk`, `rst_n`, `a`, `b` and `p` for 14×7, with the 2-cycle latency marked by markers A and B

The waveform shows the example 14×7. Values are in hexadecimal, so `E` = 14 and `62` = 98. The input `b` changes to 7 at 4640 ns while `a` is 14. Marker A (4645 ns) is the first rising edge, where the inputs are stored, and marker B (4655 ns) is the second rising edge, where `p` becomes 98. This confirms the 2-cycle latency.

2 Part B - Logic Synthesis with Yosys

2.1 Synthesis Results

The RTL was synthesized with Yosys using the SkyWater SKY130 standard cell library `sky130_fd_sc_hd` (typical corner, 25 °C, 1.80 V). The design was flattened into a single module, so the 12 full adders were merged into one block of logic. The results below are taken from `synth/mult_array_stat.rpt`.

Metric	**Value**
Total number of cells	68
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	785.754
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	50.96 %
3 most-used cell types	`dfrtp_1` (16), `nand2_1` (8), `and2_0` and `xnor2_1` (6 each, tied)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 1: Part B synthesis results

The 16 flip-flops match the design exactly: 4 for input `a`, 4 for input `b` and 8 for the product `p`. All of them were mapped to `dfrtp_1`, a flip-flop with an active-low reset, which matches the asynchronous `rst_n` in the RTL.

2.2 Gate-Level Netlist Simulation

The synthesized netlist `synth/mult_array_netlist.v` was simulated with the same testbench used in Part A (`tb/tb_mult.v`), together with the SKY130 cell models. All 256 input combinations were checked again, and the result was:

```
PASS: 256 vectors checked, 0 errors
```

This shows that synthesis did not change the behaviour of the design: the gate-level netlist gives the same product as the RTL for every input.

2.3 Report Questions

2.3.1 Q1: Logic Cell Count vs. 16 AND Gates + 12 Full Adders

Synthesis produced 68 cells in total. Removing the 16 flip-flops leaves $68 - 16 = 52$ logic cells.

This is not simply $16 + 12 = 28$, because the tool did not use a single full-adder cell for each full adder. Each full adder needs a sum output and a carry output, so the tool builds it from several smaller cells. For example, `maj3_1` produces the carry and `xor3_1`/`xnor3_1` or `xnor2_1` cells produce the sum.

The count is also not simply AND gates plus full adders, because the tool optimizes the whole design together:

- Some adders have a constant 0 input (the first carry-in of each row, and the top bit passed down from row 0), so parts of them reduce to a simple XOR or AND.

- Some AND gates are merged into larger cells such as `and4_1`, `a21oi_1` and `a22oi_1`, which do the job of several gates in one cell.
- Logic that is shared between neighbouring adders is built only once.

So the 52 logic cells are the tool's smallest-area way of building the same function, not a one-to-one copy of the 16 AND gates and 12 full adders.

2.3.2 Q3: Flip-Flop Share of Total Cell Area

The 16 flip-flops take $400.384\,\mu\text{m}^2$ out of $785.754\,\mu\text{m}^2$, which is 50.96 % of the total cell area.

This is large for two reasons:

- A flip-flop is much bigger than a simple gate. One `dfrtp_1` is $25.024\,\mu\text{m}^2$, while one `nand2_1` is about $3.75\,\mu\text{m}^2$, so one flip-flop is roughly the size of 6–7 NAND gates.
- The logic in a 4-bit multiplier is very small (only 52 cells), while the number of flip-flops is fixed by the design at 16. So the flip-flops end up taking about half of the area.

For a larger multiplier the logic grows faster than the number of flip-flops (N^2 against $4N$), so this percentage would become smaller.

3 Part C - Static Timing Analysis with OpenSTA

3.1 Finding the Maximum Frequency

Static timing analysis checks every register-to-register path against the clock period without running a simulation. For a path that launches at one flip-flop and is captured at the next, the data must arrive and settle before the capturing clock edge. This gives the setup constraint:

$$T \;\geq\; t_{clk\to Q} + t_{logic} + t_{setup}$$

The *slack* of a path is the margin left over:

$$\text{slack} = \underbrace{(T - t_{setup})}_{\text{required time}} - \underbrace{(t_{clk\to Q} + t_{logic})}_{\text{arrival time}}$$

Positive slack means the path meets timing; negative slack means it is too slow. The worst slack (WNS) belongs to the slowest path, the *critical path*. Because slack falls one-for-one as T shrinks, the smallest period that still passes is

$$T_{\min} = T - \text{WNS}, \qquad F_{\max} = \frac{1000}{T_{\min}}\ \text{MHz}$$

All runs use the typical corner (TT, 25 °C, 1.80 V), an ideal clock (no clock tree yet), and an input and output delay of 1.0 ns.

3.1.1 Run at $T = 10$ ns

At $T = 10$ ns the worst setup slack is **+7.027 ns**, so the design meets timing with a large margin. The critical path has:

- Arrival time: $t_{clk\to Q} + t_{logic} = 0.410 + 2.432 = 2.842$ ns
- Required time: $T - t_{setup} = 10.000 - 0.131 = 9.869$ ns
- Slack: $9.869 - 2.842 = 7.027$ ns

The reset path (asynchronous reset recovery check) and the input/output paths all have more slack than this, so the register-to-register path through the array sets the speed of the design. From this run:

$$T_{\min} = 10 - 7.027 = 2.973\ \text{ns} \approx 2.97\ \text{ns}$$

3.1.2 Run at $T = T_{\min}$

Re-running at $T = 2.97\,\text{ns}$ gives a worst slack of $-0.003\,\textbf{ns}$:

- Required time: $2.970 - 0.131 = 2.839\,\text{ns}$
- Arrival time: $2.842\,\text{ns}$ (unchanged, since the logic delay does not depend on the clock period)
- Slack: $2.839 - 2.842 = -0.003\,\text{ns}$

The slack is essentially zero, which confirms the calculation. The small negative value appears only because the exact $T_{\min}$ of $2.973\,\text{ns}$ was rounded down to $2.97\,\text{ns}$. A period of $2.98\,\text{ns}$ would give a slack of $+0.007\,\text{ns}$. Note that the arrival time is identical in both runs: changing the clock period only moves the required time, which is why $T_{\min} = T - \text{WNS}$ works.

3.2 Timing Results

Metric	**Value**
Worst slack at $T = 10\,\text{ns}$	$7.027\,\text{ns}$
$T_{\min}$	$2.97\,\text{ns}$
$F_{\max}$	$336.70\,\text{MHz}$
Slack at $T = T_{\min}$	$-0.003\,\text{ns}$ (≈ 0)
Critical path start point	Input register `b_r[0]`
Critical path end point	Output register `p[7]`
Number of cells on the critical path	10 (2 flip-flops + 8 logic cells)
Latency ($2 \times T_{\min}$)	$5.94\,\text{ns}$
Throughput	336.70 million results/s

Table 2: Part C static timing results (typical corner)

The multiplier has two register stages: inputs are captured at the first clock edge and the product at the second. The latency is therefore two clock periods, $2 \times 2.97 = 5.94\,\text{ns}$. Because a new pair of inputs can enter on every clock edge, the throughput is one result per cycle, which equals $F_{\max}$: 336.70 million results per second.

3.3 Critical Path

3.3.1 Start Point and End Point

- **Start point:** the input register holding `b[0]`.
- **End point:** the output register for `p[7]`, the most significant product bit, whose input is the final carry out of row 3.

This matches the structure of the array. Bit `b[0]` forms partial product pp_0, which enters the array at the very top (row 0 into row 1). Bit `p[7]` is the last carry produced at the bottom-left of the array (row 3). A signal starting at `b[0]` therefore has to travel down through every row and then along the carry chain of the last row. This is the longest route through the multiplier.

3.3.2 Cells on the Critical Path

#	Cell	Delay (ns)	Arrival (ns)	Role
1	`dfrtp_1`	0.410	0.410	Input flip-flop (clock to Q)
2	`and4_1`	0.223	0.634	Partial product generation
3	`o21ai_0`	0.195	0.828	First adder logic
4	`nand3_1`	0.140	0.969	First adder logic
5	`maj3_1`	0.380	1.349	Full-adder carry
6	`xnor3_1`	0.393	1.741	Full-adder sum
7	`maj3_1`	0.369	2.110	Full-adder carry
8	`maj3_1`	0.383	2.493	Full-adder carry
9	`maj3_1`	0.349	2.842	Full-adder carry (to `p[7]`)
10	`dfrtp_1`	setup 0.131	—	Output flip-flop

Table 3: Cells on the critical path, in order

The `maj3` cell is a 3-input majority gate, $\mathrm{maj}(a, b, c) = ab + bc + ca$, which is exactly the carry-out of a full adder. The `xnor3` cell is a 3-input XNOR, which produces the full-adder sum (the inversion is absorbed by the surrounding logic). The synthesis tool mapped most full adders onto these two cells. In total, the combinational logic takes 2.432 ns, and the flip-flop overhead ($t_{clk \to Q} + t_{setup}$) takes 0.541 ns, about 18% of $T_{\min}$.

3.3.3 Comparison with the $2N$ Full-Adder Estimate

For an N-bit array multiplier, the longest path is roughly $2N$ full-adder delays: about N adders going down through the rows, and about N more rippling along the carry chain of the last row. For $N = 4$ this predicts 8 full-adder delays.

The actual path has only 5 full-adder cells (4 `maj3` and 1 `xnor3`). The path is shorter than the estimate for the following reasons:

1. **Constant inputs simplify adders.** The first adder in every row has a carry-in of 0, and the top input to row 1 is also 0. Synthesis reduces these full adders to half adders or plain gates, which removes full-adder stages from the path.

2. **Logic restructuring.** The synthesis tool does not keep the full adders as separate blocks. It merges the AND gates and the first adder stage into a few compact gates (`and4`, `o21ai`, `nand3`).

3. **The carry path is fast.** Most of the path follows carry outputs (`maj3`), so a full adder's carry is passed straight on without first going through its slower sum output.

The $2N$ figure is therefore a structural upper estimate. It still captures the key trend: the delay grows linearly with N, because the signal must cross the rows and then ripple along the last row.

3.4 Report Questions

3.4.1 Q2: Critical Path and the $2N$ Estimate

The critical path starts at the input register for `b[0]` and ends at the output register for `p[7]`. It contains 10 cells: the launching flip-flop, eight logic cells and the capturing flip-flop. Its arrival time is 2.842 ns, and adding the 0.131 ns setup time gives $T_{\min} = 2.97$ ns and $F_{\max} = 336.7$ MHz. Only 5 of the logic cells are full-adder cells, compared with the estimate of $2N = 8$ full-adder

delays. The difference comes from synthesis simplifying the adders that have constant zero inputs, merging the first stage into compact gates, and routing the path mostly through the fast majority-gate carry outputs. The estimate is an upper bound that ignores these optimisations, but the path's shape, down through the rows and then along the final carry chain, matches the theory.

4 Part D - Physical Design: RTL to GDSII

4.1 Flow Setup

The RTL was taken to a GDSII layout with LibreLane, which runs Yosys, OpenROAD, Magic, KLayout and Netgen through synthesis, floorplan, placement, clock tree synthesis, routing, extraction and signoff. The design uses a fixed 150 μm × 150 μm die (`FP_SIZING = "absolute"`, `DIE_AREA = [0, 0, 150, 150]`).

The starting clock period was $1.2 \times T_{\min}$ from Part C, rounded up to the next 0.5 ns: $1.2 \times 2.97 = 3.564 \rightarrow 4.0$ ns.

4.2 Timing Closure: Run History

Unlike Part C, LibreLane signs off timing at nine process, voltage and temperature corners, including the slow corner (SS, 100 °C, 1.60 V), with extracted wire parasitics, a real clock tree and a 0.25 ns clock uncertainty. The worst setup slack across all corners (`timing__setup__ws` in `metrics.json`) is always set by the slow corner. Following the handout, the period was increased by 0.5 ns after every run that failed setup.

Run	Period (ns)	Worst setup slack (ns)	Slack at TT (ns)	DRC / LVS	Result
1	4.0	−1.254	1.167	0 / 0	Fail (setup)
2	4.5	−0.754	1.667	0 / 0	Fail (setup)
3	5.0	−0.254	2.167	0 / 0	Fail (setup)
4	5.5	−0.009	2.595	0 / 0	Fail (setup)
5	6.0	−0.001	2.929	0 / 0	Fail (setup)
6	6.0 *	+0.043	2.966	0 / 0	Pass (not used)
7	**6.5**	**+0.499**	**3.429**	**0 / 0**	**Pass (final)**

Table 4: LibreLane runs with 0.5 ns increments. *Run 6 used the same 6.0 ns period with the optional post-global-routing timing repair step (`RUN_POST_GRT_RESIZER_TIMING`) enabled as an experiment; the final run 7 uses only the settings required by the handout.

Runs 1 to 3 show slack rising by exactly 0.5 ns per step, so the slow-corner path was a fixed ≈ 5.25 ns. Runs 4 and 5 did not follow this pattern: with a looser clock, the area-oriented flow chose smaller, slower cells, and the slack ended just below zero both times. At 6.5 ns the critical path delay was the same as at 6.0 ns (≈ 6.0 ns), so the extra 0.5 ns appeared fully as slack. Run 6 showed that the 6.0 ns design can also pass if an extra timing repair step is enabled after routing.

4.3 Physical Design Results

Metric	Value
CLOCK_PERIOD used	6.5 ns
Number of runs needed	6 (4.0 to 6.5 ns in 0.5 ns steps), plus 1 experiment
Core area	17 759.5 μm^2 (die: 22 500 μm^2)
Cells after synthesis (Part B)	68
Cells after place and route	4822
Kinds of cells added	decap, fill, tap, I/O buffers, fanout buffers, clock buffers, hold buffer (Table 6)
Wirelength	2612 μm
Setup slack (worst corner, SS 100 °C 1.60 V)	+0.499 ns
Post-route $F_{\max}$	166.65 MHz
$F_{\max}$ from Part C	336.70 MHz
DRC errors (Magic / KLayout / router)	0 / 0 / 0
LVS errors	0

Table 5: Part D results (final run 7, CLOCK_PERIOD = 6.5 ns)

Hold timing also passes at every corner (worst hold slack +0.119 ns). The flow reports 11 max-slew warnings, all at the slow corner and all on the `rst_n` reset net, where the fanout buffers drive the reset pins of the flip-flops. These are not part of the signoff criteria and do not affect the multiplier data path.

4.4 Post-Route Maximum Frequency

$$F_{\max,\,\text{post-route}} = \frac{1000}{\text{CLOCK_PERIOD} - \text{slack}} = \frac{1000}{6.5 - 0.4993} = \frac{1000}{6.0007} \approx 166.65 \text{ MHz}$$

This is about half of the 336.70 MHz from Part C. Most of the difference comes from the corner: the same formula with the typical-corner slack (3.429 ns) gives $1000/(6.5 - 3.429) = 325.6$ MHz, only 3.3 % below Part C. The reasons are discussed in Q4 below.

4.5 Cells Added by Place and Route

Kind	Cell(s)	Count
Logic (LibreLane synthesis)	`a21o`, `xnor2`, `nand2`, ...	52
Flip-flops (LibreLane synthesis)	`dfrtp_2`	16
Decoupling capacitors	`decap_3`	4214
Filler cells	`fill_1` (211), `fill_2` (61)	272
Tap cells	`tapvpwrvgnd_1`	245
Input / output buffers	`clkdlybuf4s25_1` (9 in, 8 out)	17
Fanout buffers (`rst_n`)	`clkdlybuf4s25_1`	2
Clock tree buffers	`clkbuf_16` (1 root, 2 leaf)	3
Hold-fix buffer	`dlygate4sd3_1`	1
Total		**4822**

Table 6: Cell count after place and route, by kind

LibreLane runs its own Yosys synthesis on the RTL, so its netlist is not the same as the Part B netlist. It again has 52 logic cells and 16 flip-flops (68 in total), but uses different cell types (for

example `a21o`, `xor2` and `and3` instead of `maj3` and `xnor3`), mostly at drive strength `_2`. Place and route added $4822 - 68 = 4754$ cells. Of these, 4731 (decap, fill and tap) are physical cells with no logic function, and only 23 are buffers. The logic, flip-flop, buffer and tap cells together cover $1525\,\mu\text{m}^2$, only 8.6 % of the core; decap and fill cells fill the remaining space.

4.6 Layout Screenshots

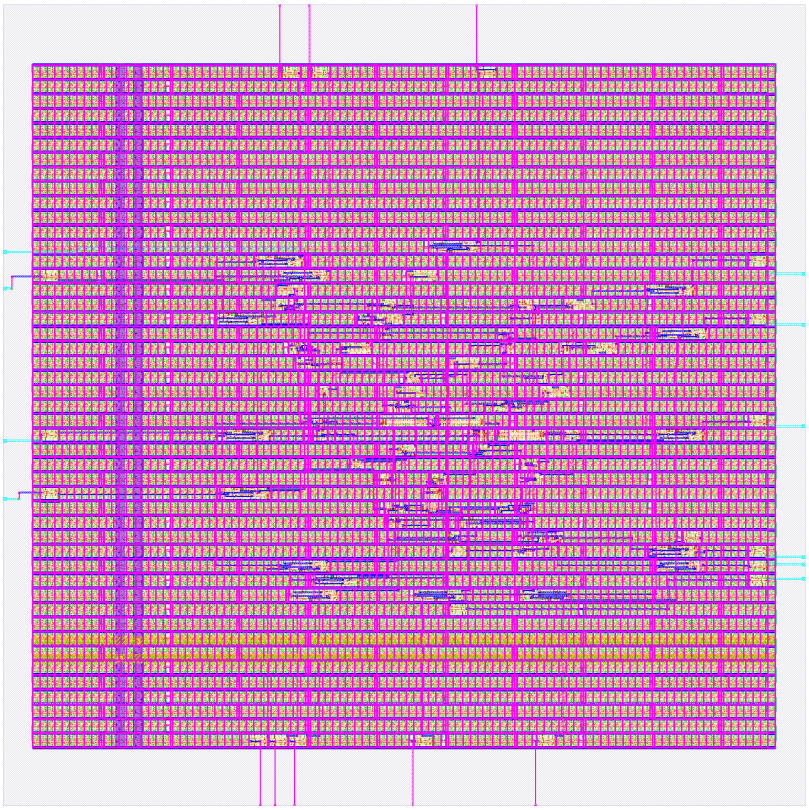


Figure 4: Final layout of the whole $150\,\mu\text{m} \times 150\,\mu\text{m}$ die in KLayout. The core is covered by standard cell rows and the power grid straps; the I/O pins are on the die edges.

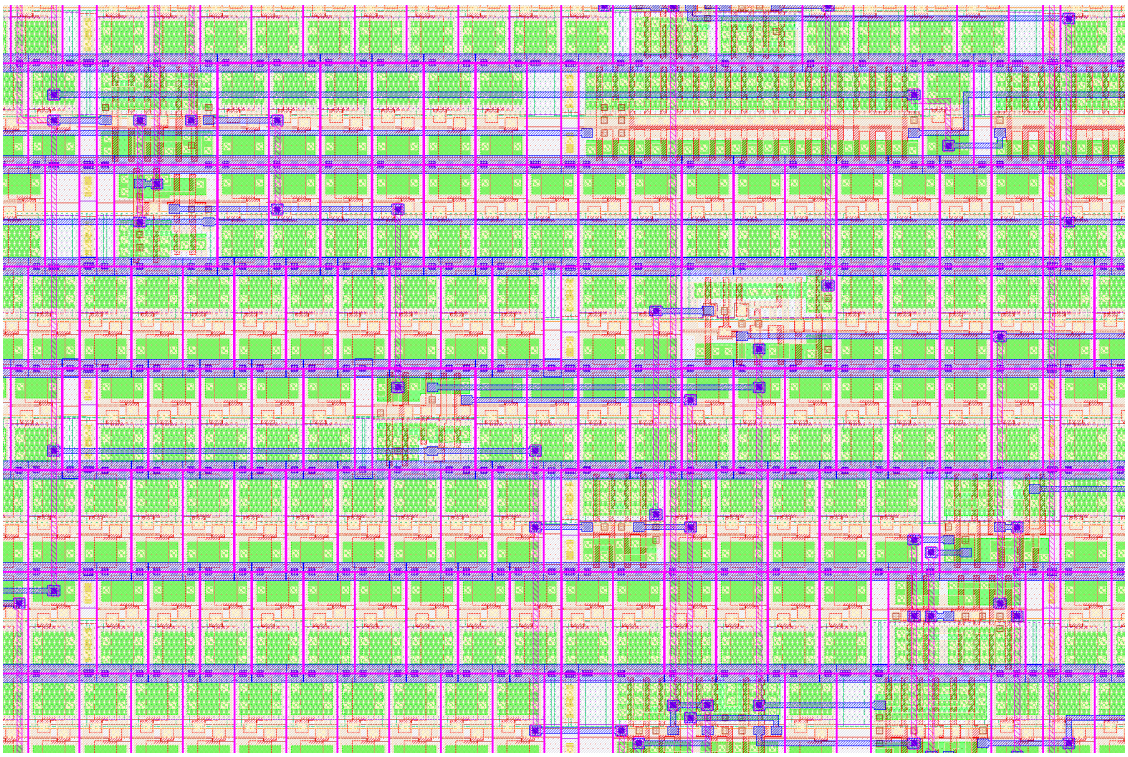


Figure 5: Zoomed view near the centre of the core: horizontal standard cell rows separated by the VPWR/VGND rails, logic cells between decap and fill cells, and horizontal and vertical metal signal wires joined by vias.

4.7 Report Questions

4.7.1 Q4: Post-Route $F_{\max}$ and Added Cells

The post-route $F_{\max}$ is 166.65 MHz, compared with 336.70 MHz in Part C. It is lower for these reasons:

- **Slow corner.** Part C used only the typical corner (TT, 25 °C, 1.80 V). LibreLane signs off at the worst corner (SS, 100 °C, 1.60 V), where every cell is much slower. This is the largest effect: at the typical corner the post-route $F_{\max}$ would be 325.6 MHz.
- **Real wires.** Part C assumed ideal wires. After routing, the 2612 μm of wire is extracted, and its resistance and capacitance add delay to every cell.
- **Real clock.** Part C used an ideal clock. After clock tree synthesis the clock arrives at each flip-flop through buffers with some skew, and the flow adds a 0.25 ns clock uncertainty margin.
- **Area-oriented cell choice.** With a loose period, the flow uses smaller, slower cells, since it only needs to meet the target period (seen in runs 4 and 5).

The flow added these kinds of cells:

- **Tap cells** (245, `tapvpwrvgnd`) connect the n-well to VPWR and the substrate to VGND at regular intervals. Without them the wells could float and the chip could suffer latch-up.
- **Decap cells** (4214, `decap_3`) are small capacitors between VPWR and VGND. They supply charge locally when many cells switch at once, which reduces supply noise (IR drop and ground bounce).

- **Filler cells** (272, `fill_1`, `fill_2`) fill the remaining gaps in each row so that the n-well, implant layers and power rails are continuous, which the DRC rules require.
- **Clock buffers** (3, `clkbuf_16`) form the clock tree so that the clock reaches all 16 flip-flops at nearly the same time with sharp edges.
- **Buffers** (20): input and output buffers isolate the internal logic from the external pins, fanout buffers let the `rst_n` signal drive all 16 flip-flop resets, and one delay buffer (`dlygate4sd3`) slows down a short path to fix a hold violation.

4.7.2 Q5: DRC and LVS

DRC (design rule check) verifies that the layout geometry follows the foundry's manufacturing rules: minimum widths and spacings of every layer, minimum enclosure of vias and contacts, minimum area, and density. **LVS** (layout versus schematic) extracts the transistors and connections from the layout and checks that they form exactly the same circuit as the gate-level netlist: same devices, same nets and same pins.

Both must pass before manufacture because they catch different failures. A DRC error means the layout may not be built as drawn: wires that are too thin or too close can break or short, which lowers yield or kills the chip. An LVS error means the layout can be built correctly but is the wrong circuit, for example a missing connection or two shorted nets, so the chip would not compute $a \times b$. Our final layout has 0 DRC errors in Magic, KLayout and the router, and 0 LVS errors with no unmatched devices, nets or pins, so the layout is both manufacturable and matches the netlist.

5 Q6 - Scaling to an 8-bit Multiplier

Using the formulas from Section 2 of the handout, for $N = 4$ and $N = 8$:

Quantity	**Formula**	**N = 4**	**N = 8**
AND gates	N^2	16	64
Full adders	$N(N-1)$	$4 \times 3 = 12$	$8 \times 7 = 56$
Flip-flops	$4N$	16	32
Critical path (full-adder delays)	$2N$	8	16

Table 7: 4-bit and 8-bit array multiplier compared

Full adders: the count grows from 12 to 56, which is $56/12 \approx 4.7$ times more. The count grows roughly as N^2, so doubling N gives about 4 times the adders.

Critical path delay: the path grows from about 8 to about 16 full-adder delays, so it doubles, because the delay grows linearly with N. Using our Part C numbers, the 4-bit logic delay is 2.432 ns and the flip-flop overhead ($t_{clk \to Q} + t_{setup}$) is 0.541 ns. Doubling the logic delay and keeping the same flip-flop overhead gives:

$$T_{\text{min, 8-bit}} \approx 2 \times 2.432 + 0.541 = 5.41 \text{ ns}, \qquad F_{\text{max, 8-bit}} \approx \frac{1000}{5.41} \approx 185 \text{ MHz}$$

compared with 2.97 ns and 336.70 MHz for the 4-bit design (typical corner). So the 8-bit multiplier needs about 4.7 times more full adders and runs at about half the clock frequency.

6 Contributions

- **Abhinav Venkata Kota:** Part A , Part C, and Questions and Report.

- **Nikhil Venkat Atkuru:** Part B , Part D , and Questions and Report.

7 Github Repo

https://github.com/DevAbhinav-23/4_Bit_Multiplier_RTL_To_GDS.git